



ANNAPOORANA ENGINEERING COLLEGE

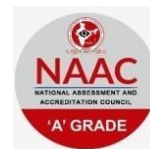
(An Autonomous Institution)

**(Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai)
Accredited by NAAC with 'A' Grade & Accredited by NBA(Civil, CSE &EEE)
2(f) & 12(B) Status as per UGC Act 1956**

ISO 9001:2015 Certified Institution

NH-47, Sankari Main Road, Periyaseeragapadi, Salem-636308, Tamil Nadu.

www.aecsale.edu.in



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL

22CS501 - Programming in Java Laboratory

Year / Semester: III / V

Batch: 2023 - 2027

Odd Semester

Prepared by

Mrs. R.UMA MAGESWARI, Assistant Professor/CSE

22CS501-PROGRAMMING IN JAVA LABORATORY

PRACTICAL EXERCISES:	30 PERIODS
1. Develop stack and queue data structures using classes and objects. 2. Develop a java application with an Employee class with Emp_name, Emp_id, Address, Mail_id, Mobile_no as members. Inherit the classes, Programmer, Assistant Professor, Associate Professor and Professor from employee class. Add Basic Pay (BP) as the member of all the inherited classes with 97% of BP as DA, 10 % of BP as HRA, 12% ofBP as PF, 0.1% of BP for staff club funds. Generate pay slips for the employees with their gross and net salary. 3. Write a Java Program to create an abstract class named Shape that contains two integers and an empty method named printArea(). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class Shape. Each one of the classes contains only the method printArea() that prints the area of the given shape. 4. Write a java program that implements a multi-threaded application that has three threads. First thread generates a random integer every 1 second and if the value is even, the second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of the cube of the number. 5. Write a program to perform file operations. 6. Develop applications to demonstrate the features of generics classes. 7. Develop applications using JavaFX controls, layouts and menus.	
TOTAL: 30 PERIODS	

EX.NO.: 1(a)	STACK DATA STRUCTURE
DATE:	

AIM:

To develop a Java program to implement stack data structure using classes and objects.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class named Stack and declare the instance variables st[], top, maxsize as private.

Step3: Use constructor to allocate memory space for the array and initialize top as -1.

Step 4: Define methods such as isFull(), isEmpty(), push(), pop() and printStack();

Step 5: Push Operation

Step 5.1: Check whether stack has some space or stack is full.

Step 5.2: If the stack has no space then display “overflow” and exit.

Step 5.3: If the stack has space then increase top by 1 to point next empty space.

Step 5.4: Add element to the new stack location, where top is pointing.

Step 5.5: Push operation performed successfully.

Step 6: Pop operation

Step 6.1: Check whether stack has some element or stack is empty.

Step 6.2: If the stack has no element means it is empty then display “underflow”

Step 6.3: If the stack has some element, accesses the data element at which top is pointing.

Step 6.4: Decrease the value of top by 1.

Step 6.5: Pop operation performed successfully.

Step 7: PrintStack operation

Step 7.1: Check whether stack has some element or stack is empty.

Step 7.2: If the stack has no element means it is empty then display “underflow”.

Step 7.3: If the stack has some element, traverse the array from top to bottom and display the elements.

Step 8: Define the main() method

Step 9: Create object of Stack class.

Step 10: Display the menu and get the user choice and invoke appropriate method.

Step 11: Stop the program.

PROGRAM:

Stack.java

```
import java.util.Scanner;
public class Stack
{
    private int maxsize, top;
    private int[] st;
    public Stack(int size)
    {
        maxsize = size;
        st = new int[maxsize];
        top = -1;
    }
    boolean isEmpty()
    {
        return top == -1;
    }
    boolean isFull()
    {
        return top == maxsize - 1;
    }
    public void push(int element)
    {
        if(isFull())
            System.out.println("Overflow");
        else
            st[++top] = element;
    }
    public int pop()
    {
        if(isEmpty())
        {
            System.out.println("UnderFlow");
            return (-1);
        }
        return (st[top--]);
    }
    public void printStack()
    {
        System.out.println("Stack Elements:");
    }
}
```

```

        for (inti = top; i>=0; i--)
            System.out.println(st[i]);
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter stack size");
        int size=sc.nextInt();
        Stack obj = new Stack(size);
        while (true)
        {
            System.out.println("\nSTACK\n*****\n1.PUSH\n2.POP\n3.Display\n4.EXIT\nEnter your choice");
            intch = sc.nextInt();
            switch (ch)
            {
                case 1:
                    System.out.println("Enter Element");
                    int n = sc.nextInt();
                    obj.push(n);
                    break;
                case 2:
                    System.out.printf("Poped element is %d", obj.pop());
                    break;
                case 3:
                    obj.printStack();
                    break;
                case 4:
                    System.exit(0);
                default:
                    System.out.println("Wrong option");
            }
        }
    }
}

```

OUTPUT:

```

D:\Java\CS3381>java Stack
Enter stack size
5

```

```

STACK
*****
1.PUSH

```

2.POP
3.Display
4.EXIT
Enter your choice
1
Enter Element
12

STACK

1.PUSH
2.POP
3.Display
4.EXIT
Enter your choice
1
Enter Element
34

STACK

1.PUSH
2.POP
3.Display
4.EXIT
Enter your choice
1
Enter Element
56

STACK

1.PUSH
2.POP
3.Display
4.EXIT
Enter your choice
1
Enter Element
78

STACK

1.PUSH
2.POP
3.Display
4.EXIT
Enter your choice
3
Stack Elements:

78
56
34
12

STACK

1.PUSH
2.POP
3.Display
4.EXIT

Enter your choice

2

Poped element is 78

STACK

1.PUSH
2.POP
3.Display
4.EXIT

Enter your choice

3

Stack Elements:

56
34
12

STACK

1.PUSH
2.POP
3.Display
4.EXIT

Enter your choice

4

D:\Java\CS3381>

RESULT:

Thus, the Java program to implement stack data structure using classes and objects has developed and executed successfully.

EX.NO.: 1(b)	QUEUE DATA STRUCTURE
DATE:	

AIM:

To develop a Java program to implement queue data structure using classes and objects.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class named Queue and declare the instance variables items[], front, rear, maxsize as private.

Step3: Use constructor to allocate memory space for the array and initialize front as -1 and rear as -1.

Step 4: Define methods such as isFull(), isEmpty(), enqueue(), dequeue() and display();

Step 5: Enqueue Operation

Step 5.1: Check whether queue has some space or queue is full.

Step 5.2: If the queue has no space then display “overflow” and exit.

Step 5.3: If the queue has space then increase rear by 1 to point next empty space.

Step 5.4: Add element to the new location, where rear is pointing.

Step 5.5: Enqueue operation performed successfully.

Step 6: Dequeue operation

Step 6.1: Check whether queue has some element or queue is empty.

Step 6.2: If the queue has no element means it is empty then display “underflow”

Step 6.3: If the queue has some element, accesses the data element at which front is pointing.

Step 6.4: Incrementthe value of front by 1.

Step 6.5: Dequeue operation performed successfully.

Step 7: Display operation

Step 7.1: Check whether queue has some element or queue is empty.

Step 7.2: If the stack has no element means it is empty then display “underflow”.

Step 7.3: If the stack has some element, traverse the array from front to rear and display the elements.

Step 8: Define the main() method

Step 9: Create object of Queue class.

Step 10: Display the menu and get the user choice and invoke appropriate method.

Step 11: Stop the program.

PROGRAM:

Queue.java

```
import java.util.Scanner;

public class Queue
{

    private int items[];
    private int maxsize, front, rear;
    Queue(int size)
    {
        maxsize=size;
        items = new int[size];
        front = -1;
        rear = -1;
    }
    boolean isFull()
    {
        if (front == 0 && rear ==maxsize-1)
        {
            return true;
        }
        return false;
    }
    boolean isEmpty()
    {
        if (front == -1)
            return true;
        else
            return false;
    }
    void enqueue(int element)
    {
        if (isFull())
```

```
        {
            System.out.println("Queue is full");
        }
        else
        {
            if (front == -1)
                front = 0;
            rear++;
            items[rear] = element;
            System.out.println("Inserted " + element);
        }
    }
    intdeQueue()
    {
        int element;
        if (isEmpty())
        {
            System.out.println("Queue is empty");
            return (-1);
        }
        else
        {
            element = items[front];
            if (front >= rear)
            {
                front = -1;
                rear = -1;
            }
            else
            {
                front++;
            }
            return (element);
        }
    }
}
```

```

    }
    void display()
    {
        inti;
        if (isEmpty())
        {
            System.out.println("Empty Queue");
        }
        else
        {
            System.out.println("\nFront index-> " + front);
            System.out.println("Items -> ");
            for (i = front; i<= rear; i++)
                System.out.print(items[i] + " ");
            System.out.println("\nRear index-> " + rear);
        }
    }
}

public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter queue size");
    int size=sc.nextInt();
    Queue obj = new Queue(size);
    while (true)
    {
        System.out.println("\nQUEUE\n*****\n1.ENQUEUE\n2.DEQUEUE\n3.DISPLAY\n4.EXIT\nEnter your choice");
        intch = sc.nextInt();
        switch (ch)
        {
            case 1:
                System.out.println("Enter Element");
                int n = sc.nextInt();
                obj.enqueue(n);

```

```

        break;
    case 2:
        System.out.printf("Dequeued element is %d", obj.deQueue());
        break;
    case 3:
        obj.display();
        break;
    case 4:
        System.exit(0);
    default:
        System.out.println("Wrong option");
    }
}
}
}
}

```

OUTPUT:

D:\Java\CS3381>javac Queue.java

D:\Java\CS3381>java Queue

Enter queue size

5

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

1

Enter Element

12

Inserted 12

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

1

Enter Element

34

Inserted 34

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

1

Enter Element

56

Inserted 56

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

1

Enter Element

78

Inserted 78

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

3

Front index-> 0

Items ->

12 34 56 78

Rear index-> 3

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

2

Poped element is 12

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

3

Front index-> 1

Items ->

34 56 78

Rear index-> 3

QUEUE

1.ENQUEUE

2.DEQUEUE

3.DISPLAY

4.EXIT

Enter your choice

4

D:\Java\CS3381>

RESULT:

Thus, the Java program to implement queue data structure using classes and objects has developed and executed successfully.

EX.NO.: 2	PAYSLIP GENERATION USING INHERITANCE
DATE:	

AIM:

To develop a java application to generate pay slip for different category of employees using the concept of inheritance.

ALGORITHM:

Step 1: Start the program.

Step 2: Create the class **Employee** with name, Empid, address, mailid, mobileno as fields.

Step 3: Inherit the classes **Programmer**, **AssistantProfessor**, **AssociateProfessor** and **Professor** from employee class.

Step 4: Add Basic Pay (BP) as the member of all the inherited classes.

Step 5: Calculate DA as 97% of BP, HRA as 10% of BP, PF as 12% of BP, Staff club fund as 0.1% of BP.

Step 6: Calculate gross salary and net salary.

Grosssal=BP+HRA+DA

NetSal=GrossSal-(PF+Staff Club Fund)

Step 7: Create a test class **PaySlip**. Read employee details, choice and basic pay from user.

Step 8: Based on user choice and input create the object and invoke the necessary methods to display the Payslip.

Step 9: Stop the program.

PROGRAM:**PaySlip.java**

```
import java.util.Scanner;
class Employee
{
    String Emp_name, Mail_id, Address, Emp_id, Mobile_no;
    double BP, GP, NP, DA, HRA, PF, CF;
    Scanner get = new Scanner(System.in);
    Employee()
    {
        System.out.println("Enter Name of the Employee:");
        Emp_name = get.nextLine();
```

```

        System.out.println("Enter Address of the Employee:");
        Address = get.nextLine();
        System.out.println("Enter ID of the Employee:");
        Emp_id = get.nextLine();
        System.out.println("Enter Mobile Number:");
        Mobile_no = get.nextLine();
        System.out.println("Enter E-Mail ID of the Employee :");
        Mail_id = get.nextLine();
    }
    void display()
    {
        System.out.println("Employee Name: "+Emp_name);
        System.out.println("Employee Address: "+Address);
        System.out.println("Employee ID: "+Emp_id);
        System.out.println("Employee Mobile Number: "+Mobile_no);
        System.out.println("Employee E-Mail ID"+Mail_id);
        DA=BP*0.97;
        HRA=BP*0.10;
        PF=BP*0.12;
        CF=BP*0.01;
        GP=BP+DA+HRA;
        NP=GP-PF-CF;
        System.out.println("Basic Pay :"+BP);
        System.out.println("Dearness Allowance : "+DA);
        System.out.println("House Rent Allowance :"+HRA);
        System.out.println("Provident Fund :"+PF);
        System.out.println("Club Fund :"+CF);

        System.out.println("Gross Pay :"+GP);
        System.out.println("Net Pay :"+NP);
    }
}
class Programmer extends Employee
{
    Programmer()
    {
        System.out.println("Enter Basic pay of the Programmer:");
        BP = get.nextFloat();
    }
    void display()
    {
        System.out.println("======"+"\\n"+"Programmar  
Pay Slip"+"\\n"+"======"+"\\n");
        super.display();
    }
}
class AssistantProfessor extends Employee
{
    AssistantProfessor()
    {

```



```

        System.out.println("Enter Basic pay of the Assistant Professor:");
        BP = get.nextFloat();
    }
    void display()
    {
        System.out.println("======"+"\\n"+"Assistant ProfessorPay Slip"+"\\n"+"======"+"\\n");
        super.display();
    }
}
class AssociateProfessor extends Employee
{
    AssociateProfessor()
    {
        System.out.println("Enter Basic pay of the Professor:");
        BP = get.nextFloat();
    }
    void display()
    {
        System.out.println("======"+"\\n"+"Associate Professor Pay Slip"+"\\n"+"======"+"\\n");
        super.display();
    }
}
class Professor extends Employee
{
    Professor()
    {
        System.out.println("Enter Basic pay of the Professor:");
        BP = get.nextFloat();
    }
    void display()
    {
        System.out.println("======"+"\\n"+"Professor Pay Slip"+"\\n"+"======"+"\\n");
        super.display();
    }
}
class Payslip
{
    public static void main(String[] args)
    {
        char ans;
        Scanner sc = new Scanner(System.in);
        do
        {
            System.out.println("Main Menu");
            System.out.println("1. Programmer \\n2. Assistant Professor \\n3. Associate Professor \\n4. Professor");
            System.out.println("Enter your choice: ");

```

```

        int choice=sc.nextInt();
        switch(choice)
        {
            case 1:
                Programmer p=new Programmer();
                p.display();
                break;
            case 2:
                AssistantProfessorap=new AssistantProfessor();
                ap.display();
                break;
            case 3:
                AssociateProfessor asp=new AssociateProfessor();
                asp.display();
                break;
            case 4:
                Professor PR=new Professor();
                PR.display();
                break;
        }
        System.out.println("Do you want to go to Main Menu?(y/n): ");
        ans=sc.next().charAt(0);
        }while(ans=='y'||ans=='Y');
        sc.close();
    }
}

```

OUTPUT:

Main Menu

1. Programmer

2. Assistant Professor

3. Associate Professor

4. Professor

Enter your choice:

1

Enter Name of the Employee:

Raja

Enter Address of the Employee:

Chennai

Enter ID of the Employee:

12345

Enter Mobile Number:

9876543210

Enter E-Mail ID of the Employee :

abc@xyz.com

Enter Basic pay of the Programmer:

56000

=====

Programmar Pay Slip

=====

Employee Name: Raja

Employee Address: Chennai

Employee ID: 12345

Employee Mobile Number: 9876543210

Employee E-Mail IDabc@xyz.com

Basic Pay :56000.0

Dearness Allowance : 54320.0

House Rent Allowance :5600.0

Provident Fund :6720.0

Club Fund :560.0

Gross Pay :115920.0

Net Pay :108640.0

Do you want to go to Main Menu?(y/n):

n

RESULT:

Thus, the Java application to generate pay slip for different category of employees was implemented using inheritance and the program was executed successfully.

EX.NO.:3	ABSTRACT CLASS IMPLEMENTATION
DATE:	

AIM:

To write a Java program to calculate the area of rectangle, circle and triangle using the concept of abstract class.

ALGORITHM:

Step 1: Start the program.

Step 2: Create an abstract class named shape that contains two integers and an empty method named printArea().

Step 3: Create three classes named rectangle, triangle and circle such that each one of the classes extends the class Shape.

Step 4: Each of the inherited class from shape class should provide the implementation for the method printArea().

Step 5: In printArea() method get the input from user and calculate the area of rectangle, circle and triangle.

Step 6: In the AbstractArea, create the objects for the three inherited classes and invoke the methods and display the area values of the different shapes.

Step 7: Stop the program.

PROGRAM:**AbstractArea.java**

```
import java.util.*;
abstract class Shape
{
    int a,b;
    abstract void printArea();
}
class Rectangle extends Shape
{
    void printArea()
    {
        System.out.println("\t\tCalculating Area of Rectangle");
        Scanner input=new Scanner(System.in);
        System.out.print("Enter length: ");
        a=input.nextInt();
        System.out.print("\n\nEnter breadth: ");
```

```

        b=input.nextInt();
        int area=a*b;
        System.out.println("Area of Rectangle: "+area);
    }
}
class Triangle extends Shape
{
    void printArea()
    {
        System.out.println("\t\tCalculating Area of Triangle");
        Scanner input=new Scanner(System.in);
        System.out.print("Enter height: ");
        a=input.nextInt();
        System.out.println("\nEnter breadth: ");
        b=input.nextInt();
        double area=0.5*a*b;
        System.out.println("Area of Triangle: "+area);
    }
}
class Circle extends Shape
{
    void printArea()
    {
        System.out.println("\t\tCalculating Area of Circle");
        Scanner input=new Scanner(System.in);
        System.out.print("Enter radius: ");
        a=input.nextInt();
        double area=3.14*a*a;
        System.out.println("Area of Circle: "+area);
    }
}
class AbstractArea
{
    public static void main(String[] args)
    {
        Shape obj;
        obj=new Rectangle();
        obj.printArea();
        obj=new Triangle();
        obj.printArea();
        obj=new Circle();
        obj.printArea();
    }
}

```

OUTPUT:

Calculating Area of Rectangle

Enter length: 10

Enter breadth: 20

Area of Rectangle: 200

Calculating Area of Triangle

Enter height: 34

Enter breadth: 56

Area of Triangle: 952.0

Calculating Area of Circle

Enter radius: 23

Area of Circle: 1661.06

RESULT:

Thus, the Java program to calculate the area of rectangle, circle and triangle using the concept of abstract class was developed and executed successfully.

EX.NO.: 4	MULTITHREADING IMPLEMENTATION
DATE:	

AIM:

To write a java program to implement a multi-threaded application.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class **even** which implements first thread that computes the square of the number.

Step 3: The `run()` method implements the code to be executed when thread gets executed.

Step 4: Create a class **odd** which implements second thread that computes the cube of the number.

Step 5: Create a third thread that generates random number. If the random number is even, it displays the square of the number. If the random number generated is odd, it displays the cube of the given number.

Step 6: The Multithreading is performed and the task switched between multiple threads.

Step 7: The `sleep()` method makes the thread to suspend for the specified time.

Step 8: Stop the program.

PROGRAM:**MultiThread.java**

```
import java.util.*;
class even implements Runnable
{
    public int x;
    public even(int x)
    {
        this.x = x;
    }
    public void run()
    {
        System.out.println("New Thread "+ x +" is EVEN and Square of "+ x + " is: "
            + x * x);
    }
}
class odd implements Runnable
{
    public int x;
```

```

        public odd(int x)
        {
            this.x = x;
        }
        public void run()
        {
            System.out.println("New Thread "+ x +" is ODD and Cube of " + x + " is: " +
                               x * x * x);
        }
    }
    class A extends Thread
    {
        public void run()
        {
            int num = 0;
            Random r = new Random();
            try
            {
                for (inti = 0; i < 5; i++)
                {
                    num = r.nextInt(100);
                    System.out.println("Main Thread and Generated Number is " +
                                       num);
                    if (num % 2 == 0)
                    {
                        Thread t1 = new Thread(new even(num));
                        t1.start();
                    }
                    else
                    {
                        Thread t2 = new Thread(new odd(num));
                        t2.start();
                    }
                    Thread.sleep(1000);
                    System.out.println("-----");
                }
            }
            catch (Exception ex)
            {
                System.out.println(ex.getMessage());
            }
        }
    }
}
public class MultiThread

```



```

{
    public static void main(String[] args)
    {
        A a = new A();
        a.start();
    }
}

```

OUTPUT:

```

Main Thread and Generated Number is 33
New Thread 33 is ODD and Cube of 33 is: 35937
-----
Main Thread and Generated Number is 31
New Thread 31 is ODD and Cube of 31 is: 29791
-----
Main Thread and Generated Number is 25
New Thread 25 is ODD and Cube of 25 is: 15625
-----
Main Thread and Generated Number is 43
New Thread 43 is ODD and Cube of 43 is: 79507
-----
Main Thread and Generated Number is 14
New Thread 14 is EVEN and Square of 14 is: 196
-----

```

RESULT:

Thus, the java program to implement multithreaded application was executed successfully.

EX.NO.: 5	FILE OPERATIONS
DATE:	

AIM:

To write a java program to copy the contents of one file to another file using file operations.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class FileCopy. Get the source and destination file names from the user.

Step 3: Create object of FileInputStream by passing the source file name.

Step 4: Using read() method read the contents of the file till end of the file.

Step 5: Create object of FileOutputStream by passing the second file to the constructor.

Step 6: Use write() method to write the contents to the destination file.

Step 7: Close the file input and output stream using close() method.

Step 8: Display the contents of both files.

Step 9: Stop the program.

PROGRAM:

FileCopy.java

```
import java.io.*;
class CopyFile
{
    public static void main(String args[]) throws IOException
    {
        inti;
        FileInputStream fin = null;
        FileOutputStreamfout = null;
        if(args.length != 2)
        {
            System.out.println("Usage: CopyFile from to");
            return;
        }
        System.out.println("Displaying contents of "+ args[0]+"\\n");
        try
        {
            fin = new FileInputStream(args[0]);
            do
```

```

        {
            i = fin.read();
            if(i != -1)
                System.out.print((char) i);
        } while(i != -1);
    }
    catch(IOException e)
    {
        System.out.println("Error Reading File");
    }
    finally
    {
        try
        {
            fin.close();
        }
        catch(IOException e)
        {
            System.out.println("Error Closing File");
        }
    }
    System.out.println("\nCopying contents of " + args[0] + "to " + args[1] + "\n");
    try
    {
        fin = new FileInputStream(args[0]);
        fout = new FileOutputStream(args[1]);
        do
        {
            i = fin.read();
            if(i != -1) fout.write(i);
        } while(i != -1);
    }
    catch(IOException e)
    {
        System.out.println("I/O Error: " + e);
    }
    finally
    {
        try
        {
            if(fin != null)
                fin.close();
        }
        catch(IOException e2)

```

```

        {
            System.out.println("Error Closing Input File");
        }
        try
        {
            if(fout != null)
                fout.close();
        }
        catch(IOException e2)
        {
            System.out.println("Error Closing Output File");
        }
    }
    System.out.println("\nFile Copied\n");
    System.out.println("\nDisplaying contents of "+ args[1]+" \n");
    try
    {
        fin = new FileInputStream(args[1]);
        do
        {
            i = fin.read();
            if(i != -1)
                System.out.print((char) i);
        } while(i != -1);
    }
    catch(IOException e)
    {
        System.out.println("Error Reading File");
    }
    finally
    {
        try
        {
            fin.close();
        }
        catch(IOException e)
        {
            System.out.println("Error Closing File");
        }
    }
}
}

```

OUTPUT:

```
R:\oop>javac CopyFile.java  
R:\oop>java CopyFile FIRST.txt SECOND.txt
```

Displaying contents of FIRST.txt

To use this program, specify the name of the source file and the destination file. For example, to copy a file called FIRST.TXT to a file called SECOND.TXT, use the following command line.

```
java CopyFile FIRST.TXT SECOND.TXT
```

Copying contents of FIRST.txt to SECOND.txt

File Copied

Displaying contents of SECOND.txt

To use this program, specify the name of the source file and the destination file. For example, to copy a file called FIRST.TXT to a file called SECOND.TXT, use the following command line.

```
java CopyFile FIRST.TXT SECOND.TXT
```

```
R:\oop>
```

RESULT:

Thus, the java program to copy the contents of one file to another file using file was written, executed and verified.

EX.NO.: 6	GENERIC CLASS
DATE:	

AIM:

To write a java program to find the maximum and minimum value from the given type of elements using ageneric function.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class **Myclass** to implement generic class and generic methods.

Step 3: Get the set of the values belonging to specific data type.

Step 4:Create the objects of the class to hold integer, character and double values.

Step 5:Create the method to compare the values and find the maximum value stored in the array.

Step 6: Invoke the method with integer, character, double and string values. The output will be displayedbased on the data type passed to the method.

Step 7:Stop the program.

PROGRAM:**GenericsDemo.java**

```
class MyClass<T extends Comparable<T>>
{
    T[] vals;
    MyClass(T[] obj)
    {
        vals = obj;
    }
    public T min()
    {
        T v = vals[0];
        for(int i=1; i<vals.length; i++)
            if(vals[i].compareTo(v) < 0)
                v = vals[i];
        return v;
    }
    public T max()
    {
        T v = vals[0];
        for(int i=1; i<vals.length;i++)
            if(vals[i].compareTo(v) > 0)
                v = vals[i];
    }
}
```

```

        return v;
    }
}
class GenericsDemo
{
    public static void main(String args[])
    {
        Integer num[]={10,2,5,4,6,1};
        Character ch[]={ 'v','p','s','a','n','h' };
        Double d[]={20.2,45.4,71.6,88.3,54.6,10.4};
        String str[]= {"hai","how","are","you"};
        MyClass<Integer>iob = new MyClass<Integer>(num);
        MyClass<Character> cob = new MyClass<Character>(ch);
        MyClass<Double>dob = new MyClass<Double>(d);
        MyClass<String>sob=new MyClass<String>(str);
        System.out.println("Max value in num: " + iob.max());
        System.out.println("Min value in num: " + iob.min());
        System.out.println("Max value in ch: " + cob.max());
        System.out.println("Min value in ch: " + cob.min());
        System.out.println("Max value in d: " + dob.max());
        System.out.println("Min value in d: " + dob.min());
        System.out.println("Max value in str: " + sob.max());
        System.out.println("Min value in str: " + sob.min());
    }
}

```

OUTPUT:

```

Max value in num: 10
Min value in num: 1
Max value in ch: v
Min value in ch: a
Max value in d: 88.3
Min value in d: 10.4
Max value in str: you
Min value in str: are

```

RESULT:

Thus, the Java program to find the maximum and minimum value from the given type of elements was implemented using generics and executed successfully.

EX.NO.: 7(a)	MULTIPLE CHOICE TEST QUESTION IN JAVA FX
DATE:	

AIM:

To write a program to display multiple choice test question using JavaFX.

ALGORITHM:

Step 1: Start the program

Step 2: Import the necessary packages

Step 3: Create a public class that extends the Application class.

Step 4: Override the start() method, which is found in the Application class.

Step 5: Create the button and name "Submit" on the button.

Step 6: Create radio buttons and link them all to the group question1.

Step 7: Disable the submit button by default initially.

Step 8: Add event handlers to all the radio buttons

Step 9: Stop the program.

PROGRAM:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.RadioButton;
import javafx.scene.control.ToggleGroup;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
public class MCTest extends Application
{
    @Override
    public void start(Stage primaryStage)
    {
        primaryStage.setTitle("Test Question 1");
        Label labelfirst= new Label("What is 10 + 20?");
        Label labelresponse= new Label();
        Button button= new Button("Submit");
        RadioButton radio1, radio2, radio3, radio4;
        radio1=new RadioButton("10");
```



```

radio2= new RadioButton("20");
radio3= new RadioButton("30");
radio4= new RadioButton("40");

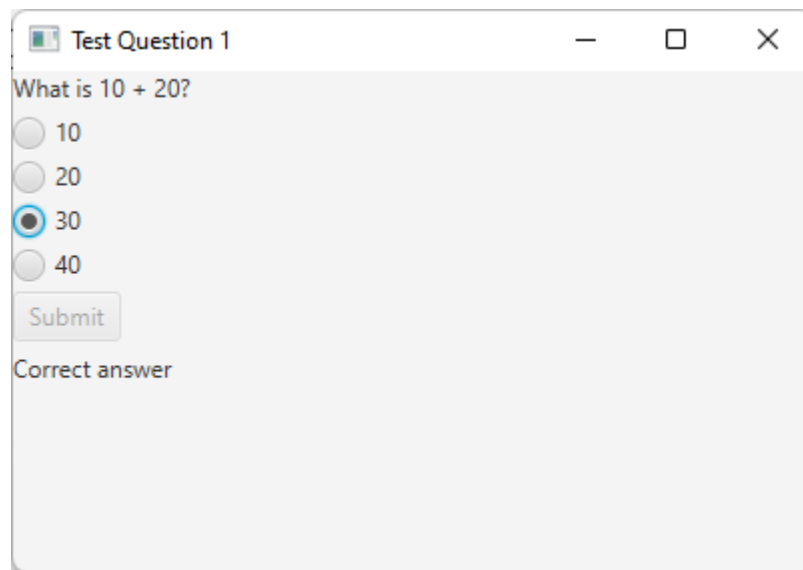
ToggleGroup question1= new ToggleGroup();
radio1.setToggleGroup(question1);
radio2.setToggleGroup(question1);
radio3.setToggleGroup(question1);
radio4.setToggleGroup(question1);

button.setDisable(true);
radio1.setOnAction(e ->button.setDisable(false) );
radio2.setOnAction(e ->button.setDisable(false) );
radio3.setOnAction(e ->button.setDisable(false) );
radio4.setOnAction(e ->button.setDisable(false) );

button.setOnAction(e ->
{
    if (radio3.isSelected())
    {
        labelresponse.setText("Correct answer");
        button.setDisable(true);
    }
    else
    {
        labelresponse.setText("Wrong answer");
        button.setDisable(true);
    }
});
VBox layout= new VBox(5);
layout.getChildren().addAll(labelfirst, radio1, radio2, radio3, radio4, button,
labelresponse);
Scene scene1= new Scene(layout, 400, 250);
primaryStage.setScene(scene1);
primaryStage.show();
}
public static void main(String[] args)
{
    launch(args);
}
}

```

OUTPUT:



The screenshot shows a Java Swing window titled "Test Question 1". Inside the window, the text "What is 10 + 20?" is displayed. Below the text are four radio button options: 10, 20, 30, and 40. The option "30" is selected, indicated by a blue dot in the center of the radio button. Below the options is a "Submit" button. At the bottom of the window, the text "Correct answer" is visible, followed by a large empty space for the correct answer.

RESULT:

Thus, the Java program for multiple choice test questions was implemented and executed successfully.

EX.NO.: 7(b)	SIMPLE EDITOR USING JAVAFX
DATE:	

AIM:

To write a program to create simple editor with menu using JavaFX.

ALGORITHM:

Step 1: Start the program

Step 2: Import the necessary packages

Step 3: Create a public class that extends the Application class.

Step 4: Override the start() method, which is found in the Application class.

Step 5: Create the menubar and add menu like File, Edit etc..

Step 6: In File menu add the menu items open, save and exit.

Step 7: In Edit menu add menu items like cut, copy and paste.

Step 8: Add event handlers to all menu items

Step 9: Create scene and add it to primary stage.

Step 10: Launch the application.

Step 11: Stop the program.

PROGRAM:

```
import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.Menu;
import javafx.scene.control.MenuBar;
import javafx.scene.control.MenuItem;
import javafx.scene.layout.VBox;
public class MenuUI extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception
    {
```

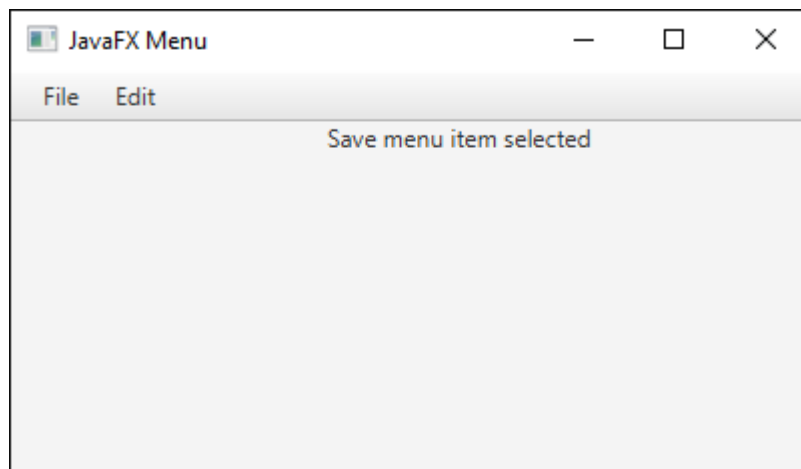
```

Menu newmenu = new Menu("File");
Menu newmenu2 = new Menu("Edit");
MenuItem m1 = new MenuItem("Open");
MenuItem m2 = new MenuItem("Save");
MenuItem m3 = new MenuItem("Exit");
MenuItem m4 = new MenuItem("Cut");
MenuItem m5 = new MenuItem("Copy");
MenuItem m6 = new MenuItem("Paste");
newmenu.getItems().add(m1);
newmenu.getItems().add(m2);
newmenu.getItems().add(m3);
newmenu2.getItems().add(m4);
newmenu2.getItems().add(m5);
newmenu2.getItems().add(m6);
MenuBar newmb = new MenuBar();
newmb.getMenus().add(newmenu);
newmb.getMenus().add(newmenu2);
Label l = new Label("\t\t\t\t\t" + "You have selected no menu items");
EventHandler<ActionEvent> event = new EventHandler<ActionEvent>() {
    public void handle(ActionEvent e)
    {
        l.setText("\t\t\t\t\t" + ((MenuItem)e.getSource()).getText() +
            " menu item selected");
    }
};
m1.setOnAction(event);
m2.setOnAction(event);
m3.setOnAction(event);
m4.setOnAction(event);
m5.setOnAction(event);
m6.setOnAction(event);
VBox box = new VBox(newmb,l);
Scene scene = new Scene(box,400,200);
primaryStage.setScene(scene);

```

```
primaryStage.setTitle("JavaFX Menu");
primaryStage.show();
}
public static void main(String[] args) {
    Application.launch(args);
}
}
```

OUTPUT:



RESULT:

Thus, the Java program for simple editor was implemented and executed successfully.